

```

include < stdio .h >
# include < stdlib .h >
# include < string .h >
# include < unistd .h >
# include < arpa / inet .h >
# define PORT 8080
# define BUFFER_SIZE 1024
void find_student ( char * reg_no , char * response ) {
FILE * fp = fopen ( " students . txt " , " r " ) ;
if ( fp == NULL ) {
strcpy ( response , " Error _ opening _ file " ) ;
return ;
}
char file_reg [ 50 ] , name [ 50 ] , branch [ 50 ] , college
[ 100 ] ;
int found = 0 ;
while ( fscanf ( fp , "%s_%s_%s_%s" , file_reg , name ,
branch , college ) != EOF ) {
if ( strcmp ( file_reg , reg_no ) == 0 ) {
sprintf ( response , " Name : _%s_ Branch : _%s_
College : _%s" , name , branch , college ) ;
found = 1 ;
break ;
}
}
}

```

```

if (! found ) {
strcpy ( response , " Data _not _ found " );
}
fclose ( fp );
}

int main () {
int server_fd , new_socket ;
struct sockaddr_in address ;
int addrlen = sizeof ( address );
char buffer [ BUFFER_SIZE ];
char response [ BUFFER_SIZE ];
server_fd = socket ( AF_INET , SOCK_STREAM , 0 );

```

```

server_fd = socket(AF_INET, SOCK_STREAM, 0);

```

```

address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

```

```

bind(server_fd, (struct sockaddr *)&address, sizeof(address));

```

```

listen(server_fd, 5);
while (1) {
new_socket = accept(server_fd,
                    (struct sockaddr *)&address,
                    (socklen_t *)&addrlen);

```

```
memset(buffer, 0, BUFFER_SIZE);

recv(new_socket, buffer, BUFFER_SIZE, 0);
find_student ( buffer , response );
send ( new_socket , response , strlen ( response ) , 0 );
}
close ( new_socket );
close ( server_fd );
return 0;
}
```